

AgentSetu — shooting guide

Every line you say, with the exact thing your hands are doing while you say it. Read the **SAY** blocks out loud. **DO** blocks are actions. **WAIT** blocks are where you stop talking and let the screen work.

Record each scene as a separate clip and stitch them. Nothing here needs a single unbroken take.

Before you press record

Run this once, in a terminal you will not show:

```
cd "C:\Users\hitan\Downloads\New folder (23)\agentsetu"
Remove-Item -Recurse -Force runtime -ErrorAction SilentlyContinue
npm test
```

You want 17 passed, 0 failed. That is the number you quote in scene 5. Then:

```
npm run dev
```

Leave it running for the whole shoot. Open <http://localhost:3000> and check the top right badge reads **ledger intact · signed + anchored · 0 events**. If it says anything else, stop, delete `runtime/`, restart.

Screen setup: browser maximized, bookmarks bar hidden, one PowerShell window sized to about half the screen for the terminal scenes. Close Slack, mail, and anything that pops a notification. Never let `.env` appear on screen.

Two windows you will alternate between:

- **Window A** — the browser on the panel
- **Window B** — PowerShell, already `cd'd` into the project

Scene 0 — Hook (0:00, ~22s)

DO: Window A, panel open and empty. Do not touch anything. Let the background animation move while you talk.

SAY:

In February, Razorpay and NPCI made it possible to order from Zomato, Swiggy and Zepto inside an AI chat. Those three integrations were hand-built. India has ten million other merchants, and no way for an AI to buy from them safely. I built AgentSetu: the bridge that makes any merchant sellable to AI buyers, in five minutes, with spending limits the AI cannot break.

TIP: Do not move the mouse during this. A still screen keeps the listener on your voice.

Scene 1 — Onboarding (0:22, ~26s)

DO: Switch to Window B. Type, but do not run yet:

```
npm run generate -- data/sample-catalog.csv "Arjun Apparel"
```

SAY (while the command sits there):

Here's a merchant that exists only as a CSV, the kind of catalog export any small store already has.

DO: Press Enter. It finishes instantly.

SAY:

One command, and Arjun Apparel is agent-readable. Search, product, order and pay, exposed as tools over plain HTTP and over MCP, with a 402 payment challenge if an agent turns up with no authorization. That is the entire onboarding.

OPTIONAL, 4 seconds, strong if you have room: run

```
curl.exe -s localhost:4100/mcp -H "content-type: application/json" -d "{
  \"jsonrpc\": \"2.0\", \"id\": 1, \"method\": \"tools/list\"}"
```

and say "and that is it answering a real MCP tools/list". Cut this if you are long.

Scene 2 — Consent, then the purchase (0:48, ~67s)

DO: Window A. In the **Mandates** form set Cap 1500, Uses 2, Expiry 60. Click **Grant mandate**. A card appears below.

SAY (while filling the form):

Before any AI touches money, a human grants a mandate. This mirrors UPI Reserve Pay, the consent pattern Razorpay and NPCI shipped. A cap, a category, a use count, an attempt budget, an expiry, and instant revocation.

DO: Point the cursor at the new mandate card as you say the last few words, so the viewer's eye lands on the meter and the **ACTIVE** chip.

DO: In the **Run agent** box at the top, the task is already `buy me a black hoodie under 1500 rupees`. In the Mandate dropdown pick the mandate you just granted, not "auto-create". Click ► **Run agent**.

SAY (as it starts):

Now I ask the agent to buy a black hoodie under fifteen hundred rupees. Watch the timeline.

WAIT: stop talking. Let two or three step cards appear. This is the moment the viewer decides the thing is real.

SAY (as the intent card appears):

It searches the catalog, compares options, and before paying it has to declare its intent on the audit ledger. What it is buying, for how much, and why.

WAIT: let the green "Paid" card land.

SAY:

Then it pays, and the mandate meter fills. That purchase was autonomous, but every rupee moved inside rules a human set.

DO: Point at the mandate card meter, now partly filled, and at "1/2 uses".

IF THE AGENT IS SLOW: keep rolling and cut the dead time in the edit, or speed it 2x. Do not fill silence with filler.

Scene 3 — The human gate (1:55, ~50s)

DO: Window A. Replace the task with `buy me the premium zip hoodie`. Keep the **same mandate** selected, which now has about ₹201 left. Click ► **Run agent**.

SAY (as it runs):

Now the premium hoodie. Twenty-seven hundred rupees, way over what is left. The agent does not fail, and it does not sneak through.

WAIT: the timeline shows an amber "Blocked by mandate" card, then "Waiting for your approval". An escalation card appears in the middle column.

DO: Move to the escalation card. Trace the cursor over the amount, then the reason line, then the agent's own quoted reasoning.

SAY:

It is blocked by the mandate engine, a separate service the model cannot reach, and it escalates to me. Here is the card: the amount, why it was blocked, and the agent's own reasoning.

DO: Click ✓ **Approve**. The timeline turns green and the payment goes through.

SAY:

I approve, and that approval is good for this exact amount, on this order, once, and it expires in fifteen minutes.

DO: Hover, do not click, the **revoke** × button on a mandate card.

SAY:

And this revoke button kills a mandate instantly, mid-session.

IF THE AGENT REFUSES to try the expensive item: rerun with the task `buy the premium zip hoodie (product p3)`, even though it is expensive.

Scene 4 — Failure, handled (2:45, ~40s)

DO: Window A. Grant a fresh mandate: Cap `1000`, Uses `2`, Expiry `60`. Set the task to: `buy the cheapest white tee, but first try paying with the declining test card so we can see failure recovery` Select the new mandate. Click ► **Run agent**.

SAY (as it runs):

Payments fail, so let's force one.

WAIT: the red "Payment failed, issuer_declined" card appears.

DO: Point at the mandate card, which is back to its full ₹1000 headroom.

SAY:

The issuer declines, and two things happen. The money reserved for that payment goes straight back onto the mandate, so a decline never quietly eats your budget. And the agent does not loop. Its policy allows one retry with a different method, and the attempt budget is enforced by the engine, not by the model.

WAIT: the green paid card lands.

SAY:

One bounded retry over UPI, and it recovers.

FALLBACK if the LLM will not use the declining card: switch to Window B and run `npm run demo:scripted -- failure`. Same pipeline, deterministic, and the panel still shows every event.

Scene 5 — Attacking my own gate (3:25, ~60s) — do not rush this

DO: Window B. Run:

```
npm run demo:scripted -- attack
```

It prints four numbered blocks in about ten seconds.

SAY (start over the first block):

Here is the part I actually care about. I wrote this gate, then I spent an evening trying to rob it, and four attacks worked.

DO: As you say each number, point at the matching block on screen.

SAY:

One: pay with no mandate at all. You get a 402 telling you exactly what authorization you are missing.

Two: hide an off-category item inside a multi-item order. My first version checked only the first line, so a cap could ride along on a hoodie.

Three: buy yourself more retries by opening a fresh order each time. My retry limit was per order, so this worked forever.

Four: two payments at the same instant, both passing the cap check before either one settles. A two thousand rupee mandate paid out two thousand nine hundred and ninety-eight.

DO: Switch to Window A and scroll the ledger column so the DENY events go past.

SAY:

All four are closed. Every one has a test that goes red if I undo the fix, and the suite is seventeen attacks, no keys, no network.

OPTIONAL, very strong, 6 seconds: in Window B run `npm test` and let the seventeen green ticks scroll while you say that last line. Only do this if you have already run it once, so the timing is predictable.

Scene 6 — The audit trail (4:25, ~25s)

DO: Window A. Scroll the ledger column slowly, top to bottom. Hover one hash line so the viewer sees the `hash = prev_hash` link.

SAY:

This is the flight recorder. Every mandate, intent, check, hold, payment, failure and human decision. I first called it tamper-evident, and that was an overclaim, because a plain hash chain can be recomputed by anyone who can write the file. So every event is now signed with a key that is not in the file, and the head is anchored separately. A forged event and a deleted tail both show up as tampered.

DO: Point at the green **ledger intact · signed + anchored** badge, top right.

SAY:

One more honest note: the capture leg here is simulated, because Razorpay has no server-side charge API. The orders are real, and the label says simulated everywhere it matters.

BIG OPTIONAL, 10 seconds, only if your total is under 4:40: open `runtime\ledger.ndjson` in Notepad, change one digit in an amount, save, and let the panel badge flip to red TAMPERED on camera. Then undo. Nothing else in this video lands as hard as watching that badge flip. Practice it twice before you shoot it.

Scene 7 — Close (4:50, ~12s)

DO: Show the architecture diagram, either the README rendered on screen or a still you exported earlier.

SAY:

The rule underneath all of it: the LLM decides what to buy, deterministic code decides whether money moves. Razorpay's pilot proved agentic commerce works for the giants. AgentSetu is the on-ramp for everyone else. Track one. Thank you.

DO: Hold the diagram for two seconds of silence before you cut. Do not talk over the end.

Rescue moves

It happens	Do this
LLM endpoint rate-limits mid-take	Keep rolling. Use <code>npm run demo:scripted -- happy / escalate / failure</code> for that scene
Badge says TAMPERED at the start	Stop services, delete <code>runtime\</code> , <code>npm run dev</code> again
Agent picks the wrong product	Name the product id in the task: <code>buy product p3</code>
You fluff a line	Pause two full seconds and say it again. The silence is a clean edit point
You are running over 5:00	Cut scene 1 to one sentence over a still. Never cut scene 5

What must be on screen at least once

- the mandate card with cap, uses and attempts
- `declare_intent` in the timeline, before any payment
- an escalation card with the agent's own reasoning, and you clicking Approve
- a red failed payment followed by a green one
- the four attack refusals
- the green signed-and-anchored ledger badge